

10. Bölüm

Sınıflar Arası İlişkiler ve SOLID İlkeleri

10.1 Unified Modeling Language

UML(**Unified Modeling Language**), nesne yönelimli yazılım geliştirme (OOP) ile birlikte gelişen bir modelleme aracıdır. Bu nedenle 8. bölümünde açıklanan nesne yönelimli birçok kavram, izleyen sayfalarda daha da pekişecektir. Adından da anlaşılacağı üzere nesneye yönelimli problem çözmenin kalbi, model oluşturmaktır.

Model, gerçek dünyadaki karmaşık problemin esaslarını soyut olarak önümüze koyar. Buradan hareketle UML dilini, basit bir dil, gösterim ya da söz dizimi gibi algılamalıyız. UML dilinin, yazılımı nasıl geliştireceğimizi kesin olarak belirtmediği göz önüne alınmalıdır.

UML, bir grafik modelleme dili olup yazılım sistemlerini oluşturan ana elemanları (**artifact**) çeşitli şekillerden oluşacak şekilde tanımlamak için oluşturulmuş bir kurallar bütünüdür. Örnek verecek olursak, bir mimari projeye ilişkin maket ne ise yazılım için UML de aynı şeyi ifade eder. Yani UML, söz konusu yazılımın neyi yapacağını anlatan çeşitli şekillerin anlamlı bir şekilde bir araya getirilmesini sağlayan bir standarttır.

Yazılım mühendisliğiyle (**software engineering**) birlikte 1989 ve 1994 yılları arasında elliden fazla modelleme dili kullanılmaya başlanmıştır. Bu nedenle bu dönem yöntem savaşlarının yaşandığı bir dönem olarak adlandırılır. Bu yöntemlerin çoğu kendi söz dizimini oluşturmuş olmakla birlikte kullandıkları dil elemanları açısından birbirleriyle benzerlik göstermektedirler.

Doksanlı yılların ortalarında üç yöntem, daha güçlü ve yaygın hale gelmiştir. Bu yöntemlerin her biri, diğerlerinin içerdiği elemanları da içeriyordu ancak her birinin diğerine karşı çeşitli güçlü özellikleri bulunuyordu. Bu yöntemler:

- ♦ Booch yönteminin tasarım (**design**) ve kodlama (**implementation**) yönleri oldukça iyi idi. *Grady Booch*, Ada diliyle oldukça çalışmıştı ve nesne yönelimli tekniklerin Ada diline uygulanması konusunda yaptığı çalışmalarla bu konuda önemli bir oyuncu olmuştur. Booch yöntemi güçlü olmasına rağmen model, birçok bulut şeklinden oluşmasından dolayı gösterim şekli oldukça sevimsizdi.
- ♦ OMT(**Object Modeling Technique**) tekniği, *Jim Rumbaugh* tarafından geliştirilmiş olup, analiz ve veri merkezli bilgi sistemlerinin modellenmesinde oldukça iyi bir yöntemdir.
- ♦ OOSE (**Object Oriented Software Engineering**) modeli **use-case** olarak bilinen bir model olup *Ivar Jacobson* tarafından geliştirilmiştir. Use-case modeli, yazılımı yapılacak sistemin davranışlarını anlamaya yönelik gelişmiş güçlü bir tekniktir.

1994 yılında *Jim Rumbaugh* ve General Electric firmasından ayrılan *Grady Booch* birlikte Rational şirketini kurarak bir araya geldiler. Bu birlikteliğin amacı, kendi yöntemlerini bir araya getirecek yeni ve tek bir yöntem olan “Unified Method” oluşturmak idi.

1995 yılında *Ivar Jacobson* da Rational şirketine katıldı. *Ivar Jacobson*’ın use-case’ler konusundaki fikirleriyle birlikte Rational şirketi, bugün UML olarak adlandırılan, yeni bir “Unified Method” geliştirdi. Üç kişiden oluşan bu grup “Three Amigos” olarak bilinir.

Yöntem savaşlarının ardından güçlü yazılım üreticileri bir araya gelerek OMG (**Object Management Group**) adında bir şirketler birliği kurdular¹. Bu şirketler birliği 1997 yılında UML’e sahip çıkarak herkes tarafından kullanılan bir dil olmasını ve bir standart haline gelmesini sağlamıştır.

UML sadece, gerçek dünyadaki süreçlerin modelini ortaya koyan bir gösterim (**notation**) sistemidir. Süreçlerin standartlaştırılmasına yönelik bir dil değildir. UML ile ortaya çıkan modeller, süreçleri soyut (**abstract**) olarak tanımlayacaktır, ancak süreçlerin doğruluğu hakkında bir bilgi vermeyecektir. Yani, A şirketi için çalışan bir süreç, B şirketine uygulandığında felaketle sonuçlanabilir. Model, yukarıda da anlatıldığı üzere çözülecek probleme ilişkin bir soyutlamadır (**abstraction**). Etki alanı (**domain**) ise problemin yaşandığı gerçek dünyayı ifade eder.

Model, birbirlerine ileti (**message**) gönderen nesnelerden (**object**) oluşur. Bu modelde nesneler canlı (**alive**) olarak düşünülmelidir. Nesneler bir şeyler bilir (**know**) ve bildikleriyle bir şeyler yaparlar (**do**). Modelde nesneler, nesne tanımında verilen özellik (**property**) ve alanları (**field**) gösteren niteliklere (**attribute**) sahiptirler.

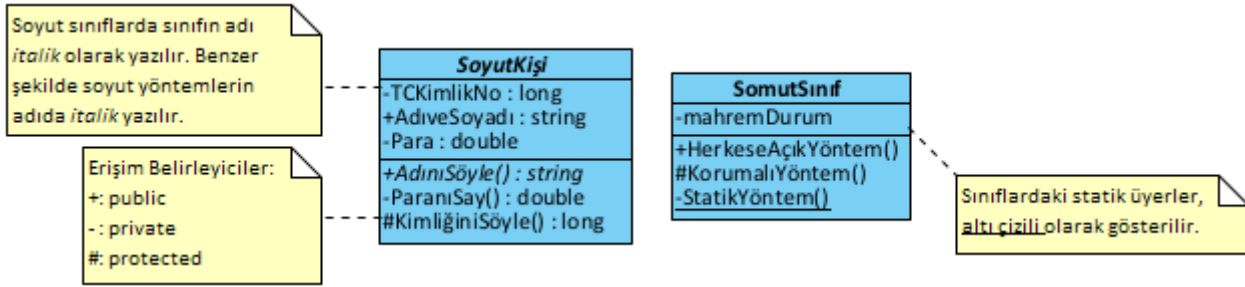
¹OMG (Object Management Group), www.omg.org

Nesnelerin gösterdikleri davranış (**behavior**) ya da geliştirdiği yöntemler (**method**) ise işlem (**operation**) olarak adlandırılır. Nesnenin özelliklerine ait o anki değerler ise nesnenin durumunu (**state**) belirler. UML, sekiz tür diyagrama sahiptir ve bu kitapta **sınıf diyagramlarına** (**class diagram**) yer verilecektir².

10.2 Sınıf Diyagramları

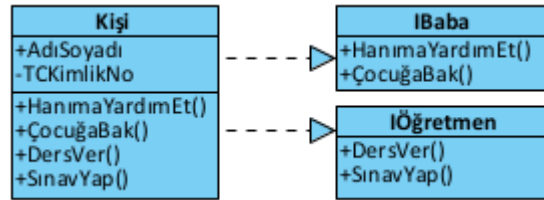
Sınıf diyagramları (**class diagram**), yazılımı geliştirilecek olan sisteme ait sınıfları (**class**) ve bu sınıflar arasındaki ilişkiyi (**relationship**) gösterir. Sınıf diyagramları durağandır (**static**). Yani sadece sınıfların bir-biriyle etkileşimini (**what interacts**) gösterir, bu etkileşimler sonucunda ne olduğunu (**what happens**) göstermez.

Sınıf diyagramlarında sınıflar, bir dikdörtgen ile temsil edilir. Bu dikdörtgen üç parçaya ayrılır. En üstteki birincisine sınıf ismi yazılır. Ortadaki ikincisinde nitelikler (**attribute**), en alttaki ve sonuncusunda ise ve işlemler (**operation**) yer alır.



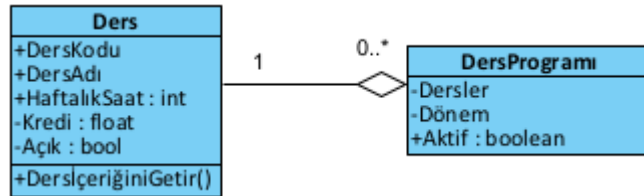
Şekil 10.1: Örnek Bir Sınıf Diyagramı

Sınıf isminin italik yazılması, sınıfın soyut sınıf (**abstract class**) olduğunu gösterir. Benzer şekilde işlemlerin yer aldığı kısımda, yöntemlerin italik yazılması halinde, ilgili yöntemin soyut yöntem (**abstract method**) olduğunu gösterir. Sınıflardaki erişim belirleyicilerin (**access modifier**), özel karakterlerle birbirinden ayrılır. Burada (-) karakteri mahrem (**private**), (+) karakteri umuma açık (**public**), (#) karakteri ise korunmalı (**protected**) olduğunu anlatmaktadır. Statik (**static**) üyeler altı çizili olarak gösterilir. **Ara yüz gerçekleştirme** (**interface realization**) işlemi, ucunda üçgen olan kesikli çizgi ile gösterilir. Üçgenin sivri köşesi gerçekleştirilen ara yüzü gösterir. Çemberler, ara yüzleri (**interface**) göstermenin bir başka yoludur. Bu durumda daha kısa ve etkili anlatıma sahip diyagramlara sahip oluruz. Bu tip basitleştirilmiş diyagramlar **lolipop diyagramı** olarak da adlandırılır. Çokluk (**multiplicity**), ilişkide bir sınıfın örnek (**instance**) sayısını gösterir. İlişkiyi gösteren



Şekil 10.2: Sınıf Diyagramlarında Arayüz Gerçekleştirme

terface realization) işlemi, ucunda üçgen olan kesikli çizgi ile gösterilir. Üçgenin sivri köşesi gerçekleştirilen ara yüzü gösterir. Çemberler, ara yüzleri (**interface**) göstermenin bir başka yoludur. Bu durumda daha kısa ve etkili anlatıma sahip diyagramlara sahip oluruz. Bu tip basitleştirilmiş diyagramlar **lolipop diyagramı** olarak da adlandırılır. Çokluk (**multiplicity**), ilişkide bir sınıfın örnek (**instance**) sayısını gösterir. İlişkiyi gösteren



Şekil 10.3: Sınıf Diyagramlarında Sınırlama ve Çokluk

çizginin sonunda bir numara olarak gösterilir. Bu numara, mümkün olabilecek örnek sayısıdır. Çokluk bir sonraki başlıkta verilen, **iş birliği** (**association**), **bileşim** (**composition**) ve **bütünleşmeye** (**aggregation**) uygulanır. Çokluk örnekleri:

- ◊ 0..1: Sıfır ya da 1 örnek.
- ◊ 0.. veya *: Örnek sayısı sınırsızdır.
- ◊ 1: Yalnızca 1 örnek
- ◊ 1..: En az bir örnek

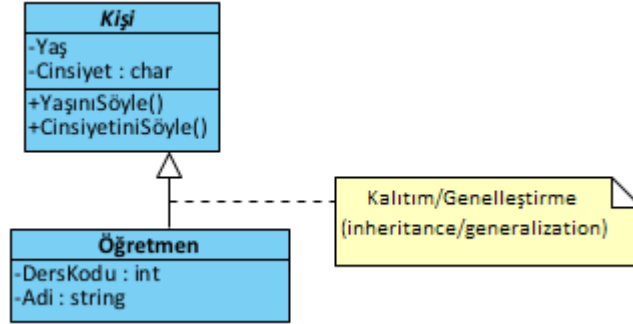
²www.togethersoftware.com/services/practical_guides/umlonlinecourse/index.html

10.3 Sınıflar Arası İlişkiler

Nesneleri imal ettiğimiz sınıflar arasında ya ilişki yoktur ya da aşağıda sıralanan altı çeşit ilişki vardır.

§10.3.1 Genelleştirme İlişkisi

Genelleştirme (**generalization**), taban sınıfla (**base class**) ile türemiş sınıf (**derived class**) arasındaki **kalıtım** (**inheritance**) ilişkisidir. Örneği 8.10 başlığında verilmiştir. Aşağıda **Kişi** sınıfı ile bu sınıftan miras alan **Öğretmen** sınıfının UML diyagramı verilmiştir.

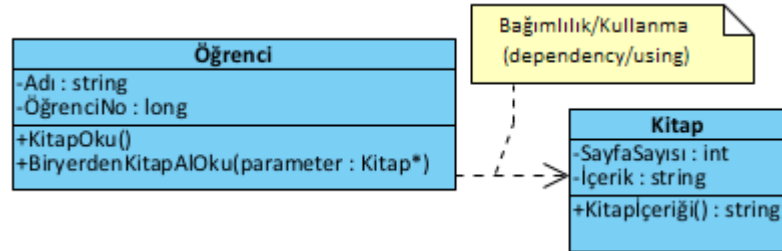


Şekil 10.4: Genelleştirme İlişkisi Örneği

§10.3.2 Bağımlılık İlişkisi

Bağımlılık (**dependency**) ya da bir başka deyişle **kullanma** (**using**), bir sınıf ile diğer arasındaki geçici ilişkidir. Burada kullanan sınıf istemci (**client**) diğer sınıf ise sağlayıcı (**supplier**) olarak adlandırılır. İki türlü bağımlılık vardır;

1. İstemci sınıf, sağlayıcıyı sınıfı **parametre olarak** kullanır (**dependency as a parameter**).
2. İstemci sınıf, sağlayıcı sınıftan **örnekleme** yapar (**dependency as an instantiation**).



Şekil 10.5: Bağımlılık İlişkisi Örneği

Aşağıda **Öğrenci-Kitap** ilişkisini gösteren bir bağımlılık örneği verilmiştir.

```

//ANA PROGRAM:
class Program
{
    static void Main(string[] args)
    {
        Öğrenci ogrenci = new Öğrenci("Ali", 12000);
        ogrenci.KitapOku(); //Kitap imal edilip kullanılıyor...
        Kitap kitap = new Kitap(350, "Kurumsal Yazılım Geliştirme");
        ogrenci.BirYerdenKitapAlOku(kitap); //Kitap imal edilip kullanılıyor...
    }
}

//SINIFLAR
public class Kitap
{
    private string _icerik;
    private int _sayfaSayisi;
    public Kitap(int sayfaSayisi, string icerik)
    {
        _sayfaSayisi = sayfaSayisi;
        _icerik = icerik;
    }
}
  
```

```

public string KitapIcerigi()
{
    return _icerik;
}
}
public class Ogrenci
{
    private string _adi;
    private int _ogrenciNo;
    public Ogrenci(string adi, int no)
    {
        _adi = adi;
        _ogrenciNo = no;
    }
    // Dependency as Instantiation (Bağımlılığın yöntem içinde oluşturulması)
    public void KitapOku()
    {
        Kitap kitap = new Kitap(150, "C# ile Nesne Yönelimli Programlama Notları");
        Console.WriteLine("Kitap imal edilip kullanılıyor...");
        string notlar = kitap.KitapIcerigi();
    }
    // Dependency Injection (Bağımlılığın parametre olarak gelmesi)
    public void BirYerdenKitapAlOku(Kitap pKitap)
    {
        if (pKitap != null)
        {
            Console.WriteLine("Kitap dışarıdan alınıp kullanılıyor...");
            string okunacak = pKitap.KitapIcerigi();
        }
    }
}

```

Bu program, nesne yönelimli programlamada sınıflar arasındaki **bağımlılık** (**dependency**) ilişkisini ve bu bağımlılığın iki farklı şekilde nasıl kurulabileceğini gösteren çok temiz bir örnektir. Programda **Öğrenci** sınıfı, işini yapabilmek (kitap okumak) için **Kitap** sınıfına ihtiyaç duyar. Örnek Program, bir nesnenin başka bir nesneye olan bağımlılığını iki ana senaryoda ele almıştır;

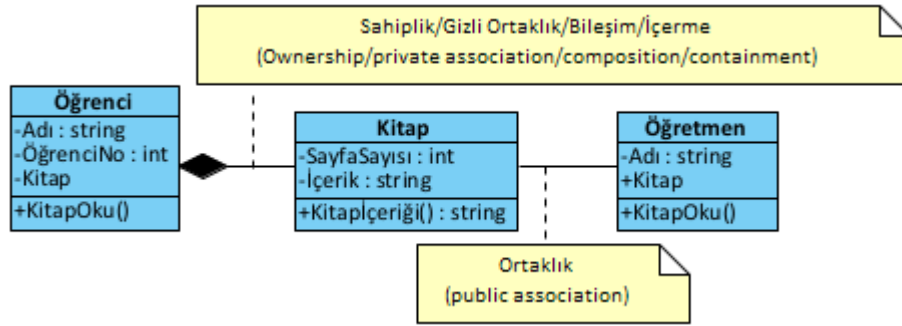
1. **Örnekleme Olarak Bağımlılık (Dependency as Instantiation):** **KitapOku()** yöntemi içinde, **Kitap** nesnesi metodun içinde **new** ile oluşturuluyor.
 - ◊ **Özellik:** **Öğrenci** nesnesi, hangi kitabın okunacağına kendisi karar verir ve kitabın yaşam döngüsünü (oluşturma ve silme) kendisi yönetir.
 - ◊ **Risk:** Bu durum sıkı **bağlılık** (**tight coupling**) yaratır. Eğer **Kitap** sınıfının yapısı (yapıcı parametreleri gibi) değişirse, **Öğrenci** sınıfının içindeki bu yöntemi de değiştirmek zorunda kalırız.
2. **Parametre Olarak Bağımlılık (Dependency as a Parameter):** **BirYerdenKitapAlOku(Kitap pKitap)** yönteminde, kitap nesnesi dışarıdan bir parametre olarak geliyor.
 - ◊ **Özellik:** Bu, modern yazılım mimarilerinde **bağımlılık enjeksiyonu** (**dependency injection**) kavramının temelidir. **Öğrenci** nesnesi, kendisine hangi kitap verilirse onu okur.
 - ◊ **Avantaj:** Daha esnek. Öğrenciye farklı kitap nesneleri (roman, ders kitabı vb.) dışarıdan kolayca gönderilebilir.

§10.3.3 Sahiplik ve Ortaklık İlişkisi

Sahiplik (**ownership**) ve **ortaklık** (**public association**) ilişkileri, iki farklı sınıf arasındaki sürekli olan bütün-parça (**whole-part**) ilişkileridir. Bunlar;

1. Sahiplik ilişkisinde, bir sınıf diğer sınıftan nesne örnekler (imal eder), kullanır ve öldürür. İlişkinin sürekli olabilmesi için kullanılan sınıftan bir alan değişkeni (**field**) mahrem (**private**) olarak tanımlanır. Bu nedenle **içerme** (**containment**), **gizli ortaklık** (**private association**) veya **bileşim** (**composition**) olarak da adlandırılır.
2. Ortaklık ilişkisinde, bir sınıf diğer sınıftan nesne örnekler (imal eder), kullanır ve öldürür. Ancak bu nesnenin dışarıdan değiştirilmesine izin verir. İlişkinin geçici olabilmesi için kullanılan sınıftan bir alan değişkeni (**field**) umuma açık (**public**) olarak tanımlanır.

Aşağıda **Kitap**, **Öğrenci** ve **Öğretmen** nesnelerinin olduğu bir ortaklık örneği verilmiştir. **Öğrenci**, **Kitap** nesnesine sahip iken **Öğretmen** nesnesi **Kitap** ile ortaklık ilişkisindedir;



Şekil 10.6: Sahiplik ve Ortaklık İlişkisi Örneği

```

//ANA PROGRAM
class Program
{
    static void Main()
    {
        // 1. Öğrenci Testi
        Öğrenci öğrenci = new Öğrenci("Ali", 12000);
        öğrenci.KitapOku();
        // 2. Öğretmen Testi
        Öğretmen öğretmen = new Öğretmen("Ilhan");
        öğretmen.KitapOku();
        // Öğretmenin kitabına mülkiyet (Property) üzerinden ulaşılabilir
        Kitap öğretmeninOkuduguKitap = öğretmen.AktifKitap;
        Console.WriteLine("Öğretmenin Okuduğu Kitaba Main İçinden Ulaşılabilir;");
        Console.WriteLine($"Okunan Kitap: {öğretmeninOkuduguKitap.Kitapİçeriği()}");
        // Kitap değiştirme işlemi (Dışarıdan enjekte etme)
        Kitap baskaKitap = new Kitap(250, "Kurumsal Yazılım Geliştirme");
        öğretmen.AktifKitap = baskaKitap;
        öğretmen.KitapOku();
    }
}

//SINIFLAR
public class Kitap
{
    private string _icerik;
    private int _sayfaSayisi;
    public Kitap(int sayfaSayisi, string icerik)
    {
        _sayfaSayisi = sayfaSayisi;
        _icerik = icerik;
    }
    public string Kitapİçeriği()
    {
        return _icerik;
    }
}

public class Öğrenci
{
    private string _adı;
    private int _öğrenciNo;
    private Kitap _kitap;
    public Öğrenci(string adı, int no)
    {
        _adı = adı;
        _öğrenciNo = no;
        // SAHİPLİK İLİŞKİSİ (Composition): Kitap, Öğrenci ile doğar.
        _kitap = new Kitap(150, "C ile Yapısal Programlama");
    }
    public void KitapOku()

```

```

{
    Console.WriteLine("Öğrenci ile birlikte imal edilen kitap kullanılıyor...");
    Console.WriteLine($"Okunan Kitap: {_kitap.Kitapİçeriği()}");
}
}
public class Öğretmen
{
    private string _adı;
    // ORTAKLIK İLİŞKİSİ (Public Association): Özellik (Property) olarak dışa açılıyor.
    public Kitap AktifKitap { get; set; }
    public Öğretmen(string adı)
    {
        _adı = adı;
        // İlk üretimde bir kitap atanıyor
        AktifKitap = new Kitap(200, "C++ ile Nesne Yönelimli Programlama");
    }
    public void KitapOku()
    {
        Console.WriteLine("Öğretmen ile birlikte imal edilen kitap kullanılıyor...");
        Console.WriteLine($"Okunan Kitap: {AktifKitap.Kitapİçeriği()}");
    }
}

```

Örnek program, nesne yönelimli programlamada nesneler arasındaki **sahiplik** (**ownership**) ve birliktelik (**association**) ilişkilerini, özellikle de mahrem (**private**) ve umuma açık (**public**) erişim düzeylerinin bu ilişkileri nasıl kısıtladığını veya esnettiğini gösterir. Programdaki temel kavramları ve teknik detayları;

1. **Sahiplik** (**ownership** / **composition** / **private association**): **Öğrenci-Kitap** Sınıfları
 - ◊ **Durum**: **Öğrenci** nesnesi yaratıldığında **Kitap** nesnesi de onun içinde otomatik olarak **new** ile oluşturuluyor.
 - ◊ **Gizlilik**: **_kitap** üyesi **private** olduğu için, ana program içinden öğrencinin kitabına erişilemez veya bu kitap değiştirilemez.
 - ◊ **Yaşam Döngüsü**: Kitap, öğrenciye bağımlıdır.
2. **Ortaklık** (**public association**): **Öğretmen-Kitap** Sınıfları
 - ◊ **Durum**: Öğretmen de bir kitapla doğar ve bu kitap dış dünyaya açılmıştır.
 - ◊ **Esneklik**: Ana programda öğretmenin okuduğu kitaba ulaşabildik ve hatta kitabını başka bir kitap nesnesiyle değiştirebildik.
 - ◊ **Bağımsızlık**: Öğretmen nesnesi hala hayattayken, ona dışarıdan bambaşka bir kitap nesnesi enjekte edilebilir.

§10.3.4 Kalıtım Yerine Kompozisyon İlkesi

Kalıtım Yerine Kompozisyon (**composition over inheritance**), nesne yönelimli programlama (OOP) dünyasında kodun daha esnek, bakımı kolay ve değişime dirençli olması için geliştirilmiş temel bir tasarım ilkesidir. Kompozisyon yukarıda anlatılan sahiplik ilişkisidir. Bu ilke, sınıflar arasındaki ilişkiyi kurarken **"is-a"** ilişkisi yerine **"has-a"** ilişkisini tercih etmemiz gerektiğini savunur.

Özellik	Kalıtım	Kompozisyon
Bağımlılık	Sıkı bağıllık (tight coupling).	Gevşek bağıllık (loose coupling).
Değişim	Üst sınıf değişirse her şey etkilenir.	Parçalar birbirinden bağımsız değişebilir.
Esneklik	Tasarım zamanında (static) belirlenir.	Çalışma anında (dynamic) değiştirilebilir.
Karmaşıklık	Çok derin hiyerarşiler kafa karıştırıcıdır.	Küçük, odaklanmış ve yönetilebilir sınıflar oluşur.

Tablo 10.1: Niçin Kompozisyon Kullanılmalı?

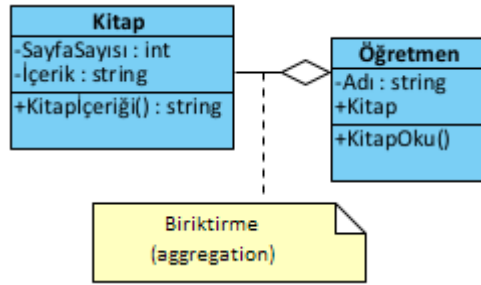
Kalıtım (**inheritance**), bir sınıfın başka bir sınıftan tüm özellik ve davranışları miras almasıdır. Genellikle katı bir hiyerarşi oluşturur. Bi bir "is-a" ilişkisidir. Örnek: "Bir Kamyon, Araç'tır". Kalıtım, "beyaz kutu" (**white-box**) yeniden kullanımıdır. Üst sınıftaki (**parent**) bir değişiklik, alt sınıfların (**children**) tamamını bozabilir. Hiyerarşi derinleştikçe sistem kırılabilir.

Kompozisyon (**composition**), bir sınıfın işlevselliğini başka bir sınıfın nesnesini kendi içinde barındırarak (ona sahip olarak) elde etmesidir. Kompozisyon bir "has-a" ilişkisidir. Örnek: "Bir Araba, Motor'a sahiptir". Kompozisyon, "siyah kutu" (**black-box**) yeniden kullanımıdır. İçerideki nesnenin nasıl çalıştığını bilmenize gerek yoktur, sadece arayüzünü kullanırsınız. Nesneler çalışma anında (**run-time**) kolayca değiştirilebilir.

Kalıtım, eğer bir sınıf, diğerinin gerçekten özelleşmiş bir versiyonuysa ve bu ilişki zamanla hiç değişmeyecekse (Örn: Daire bir Sekildir) kullanılabilir. Kompozisyon ise davranışları birleştirmek, modüler yapılar kurmak ve sistemin gelecekteki değişimlere uyum sağlamasını istiyorsanız (Örn: OdemeSistemi bir KrediKartina sahiptir) tercih edilmelidir.

§10.3.5 Biriktirme İlişkisi

Biriktirme (**aggregation**) ilişkisi, iki farklı sınıf arasındaki sürekli olan bütün-parça (**whole-part**) ilişkisidir. Ancak **sahiplik** (**ownership**) ve **ortaklık** (**public association**) ilişkilerinden farklı olarak burada nesnelerin yaşam döngüleri birbirinden ayrılmıştır.



Şekil 10.7: Biriktirme İlişkisi Örneği

```
//ANA PROGRAM
class Program
{
    static void Main()
    {
        // 1. Önce bağımsız parçalar (kitaplar) oluşturulur.
        Kitap kitap1 = new Kitap(150, "C# ile Nesne Yönelimli Programlama");
        Kitap kitap2 = new Kitap(200, "Veri Yapıları ve Algoritmalar");
        // 2. Bütün (Öğretmen) mevcut bir kitap referansı ile oluşturulur.
        Öğretmen ilhan = new Öğretmen("İlhan", kitap1);
        ilhan.KitapOku();
        // 3. Çalışma zamanında parça (kitap) kolayca değiştirilir.
        ilhan.AktifKitap = kitap2;
        Console.WriteLine("Öğretmen kitabını değiştirdi.");
        ilhan.KitapOku();
    }
}

//SINIFLAR
public class Kitap
{
    private string _icerik;
    private int _sayfaSayisi;
    public Kitap(int sayfaSayisi, string icerik)
    {
        _sayfaSayisi = sayfaSayisi;
        _icerik = icerik;
    }
    public string Kitapİçeriği() => _icerik;
}

public class Öğretmen
{
    private string _adı;

    // Biriktirme (Aggregation) ilişkisi: Kitap dışarıdan gelir.
    // Özellik (Property) kullanarak C++'taki get/set metodunu C# tarzına çeviriyoruz.
    public Kitap AktifKitap { get; set; }

    // Yapıcı Metot: Kitap olmadan öğretmen nesnesi oluşturulamaz.
    public Öğretmen(string adı, Kitap kitap)
    {

```

```

        _adı = adı;
        AktifKitap = kitap;
    }

    public void KitapOku()
    {
        if (AktifKitap != null)
        {
            Console.WriteLine($"{_adı} şu kitabı okuyor: {AktifKitap.Kitapİçeriği()}");
        }
    }
}

```

Programdaki biriktirme (**aggregation**) ilişkisi **Öğretmen** ve **Kitap** arasında zayıf bir sahiplik ilişkisidir;

- ♦ **Bağımsız Yaşam Döngüsü:** **Kitap** nesnesi, **Öğretmen** nesnesinden önce ana program içinde oluşturulur. Burada **Öğretmen** emekli olsa (nesne silinse) bile kitap nesnesi ana program içinde var olmaya devam eder.
- ♦ **Zorunlu Varlık:** **Öğretmen** yapıcısı olan `public Öğretmen(string adı, Kitap kitap)` bir kitap adresi beklediği için, sistemde bir kitap olmadan öğretmen nesnesi imal edilemez.
- ♦ **Esneklik:** **AktifKitap** özelliği sayesinde **Öğretmen** elindeki kitabı bırakıp başka bir kitabı alabilir.

10.4 Sınıf Tasarlama İlkeleri

Nesne yönelimli programlama (**object oriented programming**) kodumuzda değişim yönetimi (**change management**), gösterici kullanımı yerine referansları kullanarak güvenli kod (**safe code**) aynı kodları tekrar yazmak (**duplicate code**) yerine kodların yeniden kullanımı (**reusing**) sağlayan aşağıdaki özellikleri kullanırız;

- ♦ Kod küçüldüğü için kalıtım (**inheritance**),
- ♦ Veri ve kontrol soyutlaması (**abstraction**) yaparak daha güvenli bileşen tasarladığımız için sarmalama (**encapsulation**),
- ♦ Yazılım mimarisinin daha esnek ve genişleyebilir olması için çok biçimlilik (**polymorphism**).

Ayrıca yazılım geliştirmede;

- ♦ Zayıf bağlaşımın (**loose couple**) sağlanması için yazılımın bir noktasında olabilecek değişikliğin, diğer kısımlarda değişiklik gerektirmeyecek şekilde tasarlanması gerekir. Yani yapılan değişiklik, çorap söküğü gibi yazılımın diğer kısımlarını değiştirmemelidir.
- ♦ Aynı çağrışım kümesine ilişkin bütün bileşenler bir arada geliştirilmeli diğer kümelerle karıştırılmamalıdır (**seperation of concern**). Yani bileşenler arasında yüksek uyum (**high cohesion**) olmalıdır.

İşte yeniden kullanım, zayıf bağlaşım ve yüksek uyum sağlamak için programcı, nesne yönelimli programlama özellikleriyle birlikte temel ilkelere (**principle**) uyar. Programcı kodu tasarlariken, yazarken ve yeniden düzenlerken bu ilkeleri aklında tutarsa; Kod çok daha temiz, genişletilebilir ve test edilebilir olur. İlkelerin İngilizce baş harflerinin alınarak **SOLID ilkeleri** (**SOLID principles**) olarak isimlendirilir.

§10.4.1 Tek Sorumluluk İlkesi

Bu ilke ile SRP (**Single Responsibility Principle**) sınıflar, tasarlanırken birden fazla işten sorumlu tutulmamalıdır. *Robert C. Martin*'in vurguladığı gibi "**Bir sınıfın değişmek için tek bir nedeni (aktörü) olmalıdır**". Bir sınıfın tek bir şey yapması gerektiğini ve dolayısıyla değişmesi için tek bir nedeninin olması gerektiğini belirtir.

// 1. Veri Sorumluluğu: Sadece çalışanın temel bilgilerini tutar.

```

public class Çalışan
{
    public int TCKimlikNo { get; set; }
    public string Adı { get; set; }
    public decimal Maaş { get; set; }
}

```

// 2. Hesaplama Sorumluluğu: Maaş algoritmalarından sorumludur.

```

public class MuhasebeServisi
{
    public void ÖdemeHesapla(Çalışan işçi)
    {
        // Hesaplama mantığı buraya gelir.
        Console.WriteLine($"{işçi.Adı} için maaş hesaplandı.");
    }
}

```



```

    }
}
// 3. Veri Tabanı Sorumluluğu: Kayıt ve okuma işlemlerinden sorumludur (Repository Pattern).
public class ÇalışanDepolama
{
    public void Sakla(Çalışan işçi)
    {
        // Veritabanı kayıt mantığı.
        Console.WriteLine($"{işçi.Adı} veritabanına kaydedildi.");
    }
}
// 4. Raporlama Sorumluluğu: Çıktı formatlarından (PDF, Excel vb.) sorumludur.
public class RaporlamaServisi
{
    public void RaporÜret(Çalışan işçi)
    {
        // Rapor oluşturma mantığı.
        Console.WriteLine($"{işçi.Adı} için rapor oluşturuldu.");
    }
}

```

§10.4.2 Açık-Kapalı İlkesi

Bu ilke ile OCP (**Open Closed Principle**) sınıflar, **değişikliklere kapalı** (**closed for modification**) **eklen-tilere açık** (**open for extension**) şekilde tasarlanmalıdır. Bu şekilde değişikliklere açık genişleyebilir modüllere sahip oluruz. Değişikliklere kapalı olmak, mevcut bir sınıfın kodunu değiştirmemek anlamına gelirken, eklen-tilere açık olma yeni işlevler eklemek anlamına gelir.

```

using System;
using System.Collections.Generic;
// 1. GENİŞLEMeye AÇIK YAPI: Soyut Sınıf veya Ara yüz (Interface) olarak tanımlanabilir
// Yeni şekiller eklenebilir (Gelişime Açık)
public abstract class Şekil
{
    public abstract double AlanHesapla();
}
// 2. ÖZEL ŞEKİLLER: Mevcut yapıyı bozmadan eklenirler
public class Daire : Şekil
{
    public double Yarıçap { get; set; }
    public override double AlanHesapla() => Math.PI * Math.Pow(Yarıçap, 2);
}
public class Dikdörtgen : Şekil
{
    public double Genişlik { get; set; }
    public double Yükseklik { get; set; }
    public override double AlanHesapla() => Genişlik * Yükseklik;
}
// 3. DEĞİŞİME KAPALI SINIF: AlanHesaplayıcı
// Yeni bir şekil eklendiğinde bu sınıfın koduna dokunmamız gerekmez!
public class AlanHesaplayıcı
{
    public double ToplamAlan(IEnumerable<Şekil> Şekiller) //IEnumerable ileride anlatılacaktır.
    {
        double toplam = 0;
        foreach (var Şekil in Şekiller)
        {
            // Polimorfizm sayesinde her şekil kendi alanını hesaplar
            toplam += Şekil.AlanHesapla();
        }
        return toplam;
    }
}

```

§10.4.3 Liskov İkame İlkesi

Bu ilkede LSP (**Liskov Substitution Principle**) alt sınıflar türediği sınıfların yerine konulabilmelidir. Bu prensipten ilk olarak *Barbar Liskov* tarafından bahsedilmiştir. Alt sınıflar, türediği sınıf davranışlarıyla kullanılabilir. Burada amaç, alt sınıfların daha çok değişebileceği göz önüne alınarak değişimlerden an az şekilde etkilenmektir. 10.3 başlığındaki ilişkiler bu ilkenin temelini oluşturur.

Bu ilke der ki: "Bir programdaki nesneler, programın doğruluğunu bozmadan alt tipleriyle (**child classes**) yer değiştirebilmelidir." Yani, üst sınıf için yazılmış bir kod bloğu, o sınıftan türeyen herhangi bir alt sınıf geldiğinde de patlamadan çalışmaya devam etmelidir.

```
// 1. TEMEL SINIF: Tüm kuşların ortak özelliği (Beslenme)
public abstract class Kuş
{
    public virtual void Ye() => Console.WriteLine("Kuş yem yedi.");
}
// 2. DAVRANIŞSAL AYRIM: Sadece uçabilen kuşlar için bir sınıf veya arayüz
public abstract class UçanKuş : Kuş
{
    public abstract void Uc();
}
// 3. UYGULAMALAR: LSP'yi ihlal etmeyen doğru hiyerarşi
public class Serçe : UçanKuş
{
    public override void Uc() => Console.WriteLine("Serçe kanat çırparak uçuyor.");
}
public class Penguen : Kuş
{
    // Penguen sadece Kuş'tan türer.
    // fly() metodunu override edip hata fırlatmak (NotImplementedException) zorunda kalmaz.
}
```

Türemiş sınıftan bir nesneyi, Taban sınıftan bir nesne bekleyen herhangi bir yöntem parametre olarak geçirdiğimizde yöntemin herhangi bir tuhaf çıktı vermemesi gerektiği anlamına gelir. Bu beklenen davranıştır, çünkü kalıtım kullandığımızda alt sınıfın, üst sınıfın sahip olduğu her şeyi miras aldığını varsayabiliriz. Alt sınıf davranışı genişletir ancak asla daraltmaz. Dolayısıyla bir sınıf bu ilkeye uymadığında, tespit edilmesi zor bazı kötü hatalara yol açar.

§10.4.4 Ara Yüzleri Ayırma İlkesi

Bu ilke ile ISP (**Interface Segregation Principle**) genel amaçlı ara yüzler yerine istemciye bağlı ara yüzler tasarlanmalıdır. Yani roller, birbirinden ayrı olacak şekilde tanımlanmalıdır. Birden fazla rol tek bir rol altında olacak şekilde düşünülmemelidir. Bu nedenle buna rollerin ayrılması prensibi RSP (**Role Segregation Principle**) olarak da adlandırılır. Bu ilke der ki: "Hiçbir istemci (**client**), kullanmadığı yöntemlere bağımlı olmaya zorlanmamalıdır." Devasa ve her işi yapan "şişman" (**fat**) arayüzler yerine, daha spesifik ve küçük arayüzler oluşturmayı savunur. Eğer bir arayüz çok fazla yetenek barındırıyorsa, bu arayüzü uygulayan sınıflar ihtiyaç duymadıkları metotları da `throw new NotImplementedException();` diyerek boşu boşuna taşımak zorunda kalırlar.

```
// 1. KÜÇÜK VE ÖZEL ARAYÜZLER (Specific Interfaces)
public interface IYazdırabilir
{
    void Yazdır();
}
public interface ITarayabilir
{
    void Tara();
}
public interface IFaksLayabilir
{
    void FaksGönder();
}
// 2. SADECE YAZDIRMA YETENEĞİ OLAN CİHAZ
// Sadece ihtiyacı olan arayüzü uygular.
public class BasitYazıcı : IYazdırabilir
{
}
```

```

    public void Yazdır() => Console.WriteLine("Basit yazıcı çıktı veriyor...");
}
// 3. ÇOK FONKSİYONLU CİHAZ (All-in-One)
// İhtiyaç duyduğu tüm yetenekleri "biriktirir".
public class CokFonksiyonluYazici : IYazdirabilir, ITarayabilir, IFakslayabilir
{
    public void Yazdır() => Console.WriteLine("Yazdırılıyor...");
    public void Tara() => Console.WriteLine("Taranıyor...");
    public void FaksGönder() => Console.WriteLine("Fakslanıyor...");
}

```

§10.4.5 Bağımlılıkları Ters Çevirme İlkesi

Bu ilke DIP (**Dependency Inversion Principle**), sınıflarımızın somut sınıflara ve fonksiyonlara değil, ara yüzlere veya soyut sınıflara bağlı olması gerektiğini belirtir. Nesne yönelimli programlamanın en önemli ilkesidir. Aslında bu ilke ile açık-kapalı ilkesi doğrudan birbiriyle ilişkilidir. Bu ilke özetle şunu söyler: "Yüksek seviyeli modüller, düşük seviyeli modüllere bağımlı olmamalıdır. Her ikisi de soyutlamalara (arayüzler veya soyut sınıflar) bağımlı olmalıdır."

```

// 1. SOYUTLAMA (Interface): Her iki taraf da buna bakar
public interface IDatabase
{
    void Save(string data);
}
// 2. DÜŞÜK SEVİYELİ MODÜLLER (Implementations)
public class MySqlConnection : IDatabase
{
    public void Save(string data) => Console.WriteLine($"Veri MySQL'e kaydedildi: {data}");
}
public class PostgreSQLDatabase : IDatabase
{
    public void Save(string data) => Console.WriteLine($"Veri PostgreSQL'e kaydedildi: {data}");
}
// 3. YÜKSEK SEVİYELİ MODÜL (Business Logic)
// UserService, hangi veritabanının kullanıldığını bilmez, sadece IDatabase olduğunu bilir.
public class UserService
{
    private readonly IDatabase _database;
    // Constructor Injection: Bağımlılık dışarıdan enjekte ediliyor.
    public UserService(IDatabase database)
    {
        _database = database;
    }
    public void SaveUser(string user)
    {
        _database.Save(user);
    }
}

```

10.5 Düşün ve Çöz

- ♦ **Düşün:** UML sınıf diyagramında birliktelik (**composition**) ile biriktirme (**aggregation**) arasındaki farkı somut bir örneklerle gösterin. 'Araba-Motor' ilişkisi ile 'Üniversite-Öğrenci' ilişkisi bu iki kavramdan hangisine karşılık gelir ve neden?
- ♦ **Düşün:** Tek Sorumluluk İlkesi'nin (SRP) ihlal edildiğini nasıl anlarsınız? Bir sınıfın birden fazla 'değişim nedeni' olduğunu tespit etmenin pratik yöntemi nedir?
- ♦ **Düşün:** Açık-Kapalı İlkesi'ni (OCP) uygulamak için kalıtım mı, arayüz mi yoksa strateji deseni mi kullanmak daha uygundur? Bir vergi hesaplama sistemi örneği üzerinden tartışınız.
- ♦ **Düşün:** Liskov İkame İlkesi'ni (LSP) ihlal eden 'Kare-Dikdörtgen' problemi neden sezgisel olmayan bir ihlaldir? Bu sorunun kök nedeni tasarım mı yoksa dil sınırlaması mı?
- ♦ **Düşün:** Arayüzleri Ayırma İlkesi (ISP) neden 'şişman arayüzler'in (**fat interfaces**) sorunlu olduğunu savunur? Çok yöntemli bir arayüzü küçük arayüzlere bölmenin olumsuz bir etkisi var mı?
- ♦ **Düşün:** Bağımlılıkları Ters Çevirme İlkesi (DIP) ile bağımlılık enjeksiyonu (**dependency injection**) arasın-

daki ilişki nedir?

- ◇ **Düşün:** Sıkı bağlaşım (**tight coupling**) ile gevşek bağlaşım (**loose coupling**) arasındaki farkı bir HTTP servisi çağrısı örneğiyle açıklayınız. Arayüzler bu bağı nasıl gevşetir?
- ◇ **Çöz:** SOLID ilkelerinden Tek Sorumluluk İlkesi'ni ihlal eden bir Personel sınıfı yazınız (maaş hesaplama, veritabanı kaydetme ve e-posta gönderme aynı sınıfta). Ardından her sorumluluğu ayrı sınıflara taşıyın.
- ◇ **Çöz:** Liskov İkame İlkesi'ni ihlal eden 'Kare-Dikdörtgen' örneğini kodlayın; ihlalin neden oluştuğunu test koduyla gösterin ve tasarımı düzeltin.
- ◇ **Çöz:** Bağımlılık enjeksiyonu kullanarak bir bildirim sistemi tasarlayın: `INotificationService` arayüzü, `EmailNotification` ve `SmsNotification` uygulamaları. Müşteri sınıfı hangi bildirim sisteminin kullanılacağını bilmeden çalışabilsin.
- ◇ **Çöz:** Bir ödeme sistemi için Açık-Kapalı İlkesine uygun bir tasarım yapın: `KrediKarti`, `BankaTranferi` ve Kripto ödeme yöntemleri mevcut kodu değiştirmeden eklenebilsin.
- ◇ **Çöz:** Yazıcı, tarayıcı ve faks işlevleri olan büyük bir `IMultifunctionDevice` arayüzünü, Arayüzleri Ayırma İlkesine uygun olarak küçük arayüzlere bölün. Sadece yazdırma yapabilen bir sınıfı bu yapıya uygun olarak yazın.